CSC 3201 Module 7 Assignment - Steganography

Marc Helwee and Jaxon Bruzgul

Purpose: We implemented password-keyed least-significant-bit (LSB) steganography. The program hides either text or a PNG image inside the color-channel LSBs of an RGB cover image, then reconstructs the same placement order from the password to recover the payload.

Inputs: The cover was apple_logo_cover.png; the secret image was laptop_secret.png. The program used the required password secret.

Reproducibility: Run the submitted Python program from a directory containing the two input files. It writes stego_text.png, stego_laptop.png, and recovered_laptop.png, then prints the verification results shown below.

Summary: Both payloads were embedded and recovered successfully. The largest per-channel difference between either stego image and the cover was 1, which is the maximum expected change when only an LSB is overwritten.

Task 1 Embedder and Stego Images

The embedder first frames every payload with the required nine-byte header: four bytes of magic text STEG, one type byte (0x00 for text and 0x01 for image), and a four-byte big-endian payload length. This lets the extractor confirm that a payload exists, identify its type, and read exactly the number of payload bytes that were embedded.

To distribute the payload across the image, the program hashes the password with SHA-256, interprets the first eight digest bytes as an integer seed, and initializes NumPy's deterministic random-number generator with that seed. Its permutation of all RGB channel positions is used for both embedding and extraction. Thus, the correct password rebuilds the same order while another password reads unrelated channel positions.

For each framed payload bit, the program clears the selected channel LSB using & 0xFE and sets it to the payload bit using | bit. The modified image is saved as a PNG.



Figure 1. Original RGB cover image
Figure 2. Stego image containing the text payload.

Figure 3. Stego image containing the laptop PNG

Task 1 Results and Question 1

Verified terminal output

```
stego_text.png max pixel diff vs cover: 1
stego_laptop.png max pixel diff vs cover: 1
cover (225, 225) capacity bytes 18984 laptop payload bytes 8198
oversized payload -> payload too large for this cover
```

The two stego files retained the appearance of the cover image. The verification output measured a maximum pixel-channel difference of one for each stego image, consistent with LSB replacement. The embedder also rejected an intentionally oversized payload instead of silently truncating it.

Question 1 Why PNG rather than JPEG

PNG is lossless, so it preserves every RGB channel value and therefore preserves the LSBs that carry the hidden payload. JPEG uses lossy compression and quantization that change pixel values; even small changes to those values can overwrite the LSBs, corrupt the header or payload, and prevent correct extraction.

Task 2 Extractor and Payload Recovery

The extractor loads the stego image as RGB, derives the same password-seeded permutation, and reads the LSBs in that order. It first reads 72 bits, the nine-byte header. If the first four recovered bytes are not STEG, it returns (None, None) rather than treating random bits as a payload. Otherwise, it reads the type and length, reads the exact number of payload bits, and returns either UTF-8 text or PNG bytes.



Figure 4. Original laptop secret image.

Figure 5. Recovered laptop image.

The recovered image bytes were identical to the PNG bytes embedded by the program.

Task 2 Results and Question 2

Verified terminal output

```
recovered text: ('text', 'This is a secret')
recovered image kind: image identical to original: True
wrong password: (None, None)
```

Wrong-password behavior. With the wrong password, the generated permutation does not match the embedding order. The first 72 recovered bits therefore do not begin with the required STEG magic value, and the extractor safely returns no payload rather than decoding arbitrary data.

Question 2 Capacity and size limit

The 225 x 225 RGB cover has 225 x 225 x 3 = 151,875 color channels. Because one bit is stored in each channel, the image holds 151,875 embedded bits, or 18,984 complete bytes of total framed data. The nine-byte header counts against that capacity, so the largest raw payload that can fit is 18,975 bytes.

The text message is 16 bytes. Including its nine-byte header, it uses 25 bytes = 200 LSBs, or about 0.132% of the cover capacity. The re-encoded laptop PNG is 8,198 bytes. Including its header, it uses 8,207 bytes = 65,656 LSBs, or about 43.23% of the cover capacity. A larger payload cannot fit because there is only one LSB available per RGB channel; the capacity check rejects it before any image is written.

Interesting observation

This technique hides data from casual visual inspection but is not encryption. The password makes the embedding locations difficult to guess, but anyone who knows the password can recover the payload. Also, LSB steganography is fragile: any lossy image conversion, resizing, or editing can alter the LSBs and destroy the hidden data.

Submitted Code steganography.py

```
import hashlib, io, sys
import numpy as np
from PIL import Image

MAGIC = b"STEG"

def seed_from_password(password):
    return int.from_bytes(hashlib.sha256(password.encode("utf-8")).digest()[:8], "big")

def capacity_bytes(h, w, channels=3):
    return (h * w * channels) // 8

def bytes_to_bits(data):
    return np.unpackbits(np.frombuffer(data, dtype=np.uint8))

def bits_to_bytes(bits):
    bits = np.asarray(bits, dtype=np.uint8)
    bits = np.pad(bits, (0, -len(bits) % 8))
    return np.packbits(bits).tobytes()
```

```
def frame_payload(kind, payload):
    return MAGIC + (b"\x00" if kind == "text" else b"\x01") + len(payload).to_bytes(4, "big") + payload

def _perm(password, total):
    return np.random.default_rng(seed_from_password(password)).permutation(total)

def embed(cover_path, out_path, kind, payload, password):
    arr = np.array(Image.open(cover_path).convert("RGB"))
    flat = arr.reshape(-1).copy()
    bits = bytes_to_bits(frame_payload(kind, payload))
    if len(bits) > flat.size:
        raise ValueError("payload too large for this cover")
    pos = _perm(password, flat.size)[:len(bits)]
    flat[pos] = (flat[pos] & 0xFE) | bits
    Image.fromarray(flat.reshape(arr.shape)).save(out_path, "PNG")

def extract(stego_path, password):
    flat = np.array(Image.open(stego_path).convert("RGB")).reshape(-1)
    perm = _perm(password, flat.size)
    header = bits_to_bytes(flat[perm[:72]] & 1)
    if header[:4] != MAGIC:
        return None, None
    kind = "text" if header[4] == 0 else "image"
    n = int.from_bytes(header[5:9], "big")
    if (9 + n) * 8 > flat.size:
        return None, None
    payload = bits_to_bytes(flat[perm[72:72 + n * 8]] & 1)
    return (kind, payload.decode("utf-8")) if kind == "text" else (kind, payload)
```

Submitted Code steganography.py continued

```
if __name__ == "__main__":
    cover, pw = "apple_logo_cover.png", "secret"
    embed(cover, "stego_text.png", "text", "This is a secret".encode(), pw)
    buf = io.BytesIO(); Image.open("laptop_secret.png").convert("RGB").save(buf, "PNG")
    embed(cover, "stego_laptop.png", "image", buf.getvalue(), pw)
    a = np.array(Image.open(cover).convert("RGB")).astype(int)
    for f in ("stego_text.png", "stego_laptop.png"):
        print(f, "max pixel diff vs cover:", np.abs(np.array(Image.open(f).convert("RGB")).astype(int) - a).max())
    h, w, _ = a.shape
    print("cover", (h, w), "capacity bytes", capacity_bytes(h, w), "laptop payload bytes", len(buf.getvalue()))
    print("recovered text:", extract("stego_text.png", pw))
    k, data = extract("stego_laptop.png", pw)
    Image.open(io.BytesIO(data)).save("recovered_laptop.png")
    print("recovered image kind:", k, "identical to original:", data == buf.getvalue())
    print("wrong password:", extract("stego_text.png", "wrong"))
    try: embed(cover, "x.png", "image", bytes(h * w * 3 // 8), pw)
    except ValueError as e: print("oversized payload ->", e)
```

CODE